Lead Scraper — Interior Design & Architecture Database (Poland)

Scope v2.0 — Prepared by: Marketing Strategist, Data Scraping Engineer, IT Safety Engineer

---

1. Mission

Build a reliable, safe, and maintainable system that collects business contact data for interior designers, decorators, and architects operating in Poland. The output is clean Excel files ready for email marketing campaigns.

The system must:

- Survive increasing bot protection without getting blocked
- Never expose the host system to malicious content
- Never let scraped HTML or text pollute the agent's context window
- Produce deduplicated, verified data — not raw dumps

---

2. Team Roles & Responsibilities

Marketing Strategist

- Defines what makes a usable lead (minimum: company name + city + email OR phone)
- Sets segmentation criteria (category, region, company size, quality score)
- Validates output quality before campaigns
- Defines search terms and target categories in Polish market context

Data Scraping Engineer

- Builds source-specific scrapers (right tool for each job)
- Handles anti-detection, rate limiting, proxy rotation
- Solves the HTML-context-clogging problem (see Section 6)
- Maintains scrapers when sites change structure

IT Safety Engineer

- URL validation before any visit (VirusTotal, whois, TLS)
- Prompt injection prevention in scraped content
- Container sandboxing — scraper cannot affect host
- Data sanitization pipeline
- Credential isolation

---

3. Data Sources — Poland

3.1 API Sources (safest, most reliable)

Source	Data	Tool	Anti-bot	Cost
Google Places API	Name, address, phone, website, rating	HTTP client (node-fetch)	None (API key auth)	~$17/1000 req
Hunter.io API	Email by domain	HTTP client	None (API key)	$49/mo for 1000
Apollo.io API	Email, phone, company data, decision makers	HTTP client	None (API key)	Free tier: 10k/mo
Regon/GUS API (api.stat.gov.pl)	Official PKD code, NIP, address, legal form	HTTP client	None (public API)	Free

Scraping engineer note: APIs are the gold standard. No bot detection, structured JSON responses, predictable rate limits. Always prefer an API over scraping the same data from HTML.

3.2 Directory Sources (need scraping)

Each directory has different protection. We match the right tool to the job:

Source	Data	Tool	Protection Level

Panorama Firm (panoramafirm.pl)	Name, address, phone, website, category	HTTP + Cheerio	Low — server-rendered HTML
Pkt.pl	Yellow pages listings	HTTP + Cheerio	Low — simple HTML
Izba Architektów RP (izbaarchitektow.pl)	Architect names, license numbers, contact	HTTP + Cheerio	Low — public member list
SARP (sarp.org.pl)	Architect association members	HTTP + Cheerio	Low
SAW (saw.org.pl)	Interior designer association	HTTP + Cheerio	Low
Dobry Architekt (dobryarchitekt.pl)	Curated architect profiles	HTTP + Cheerio	Low
Archinea.pl	Architecture company directory	HTTP + Cheerio	Low
Homebook.pl	Professional profiles, portfolio	Playwright (headless)	Medium — JS-rendered, lazy loading
Oferteo.pl	Service provider profiles	Playwright (headless)	Medium — JS-rendered
Houzz.com/pl	Professional directory	Playwright + stealth + proxy	High — Cloudflare, fingerprinting

3.3 Enrichment Sources (secondary passes)

Source	Purpose	Tool
Company websites	Extract email from /kontakt /contact page	HTTP + Cheerio (targeted)
Instagram Graph API	Social presence, follower count	API (business accounts only)
Google Ads Transparency Center	Who runs ads for design keywords = active firms	HTTP scrape
Award sites (bryla.pl, muratorplus.pl)	Premium segment tagging	HTTP + Cheerio

---

4. Search Strategy

4.1 PKD Codes (official Polish business classification)

Code	Description
`71.11.Z`	Działalność w zakresie architektury
`74.10.Z`	Działalność w zakresie specjalistycznego projektowania
`71.12.Z`	Działalność w zakresie inżynierii (some overlap)

Use these for Regon/GUS API lookups — returns every registered business in the category.

4.2 Google Places Queries

```
"projektant wnętrz"         — interior designer
"architekt wnętrz"          — interior architect
"studio projektowe wnętrz"  — interior design studio
"pracownia architektoniczna" — architecture studio
"biuro architektoniczne"     — architecture office
"aranżacja wnętrz"          — interior arrangement
"dekorator wnętrz"          — interior decorator
```

4.3 City Grid (40 cities, covers >90% of market)

Tier 1 (metro, expect 200+ results each): Warszawa, Kraków, Wrocław, Poznań, Gdańsk, Łódź, Katowice, Szczecin, Lublin

Tier 2 (large cities, expect 50-150 each): Białystok, Bydgoszcz, Gdynia, Częstochowa, Radom, Toruń, Kielce, Rzeszów, Gliwice, Olsztyn, Bielsko-Biała, Opole, Zielona Góra

Tier 3 (smaller cities, expect 10-50 each): Elbląg, Płock, Tarnów, Koszalin, Kalisz, Legnica, Grudziądz, Wałbrzych, Włocławek, Tychy, Rybnik, Sosnowiec, Bytom, Dąbrowa Górnicza, Ruda Śląska, Zabrze, Chorzów, Gorzów Wlkp

---

5. Anti-Detection Strategy

5.1 Principle: Right Tool for the Right Job

Not every source needs a headless browser. Using Playwright when a simple HTTP request works is wasteful, slow, and more detectable.

```
Decision tree:

Is there an API?
  YES → Use HTTP client with API key. Done.
  NO ↓

Is the page server-rendered HTML?
  YES → Use HTTP client (got/undici) + Cheerio parser. No browser needed.
  NO ↓

Does the page require JavaScript rendering?
  YES → Is it behind Cloudflare/DataDome?
    NO  → Playwright in headless mode (no stealth needed)
    YES → Playwright + stealth plugin + residential proxy
```

5.2 Tool Stack by Protection Level

Level 0 — APIs:

```
Tool:     undici / node-fetch
Headers:  API key only
Rate:     Per API docs
Proxy:    None needed
```

Level 1 — Simple HTML (Panorama Firm, Pkt.pl, Izba Architektów):

```
Tool:     undici HTTP client + Cheerio
Headers:  Rotating user-agent (5 real Chrome UAs)
Rate:     2-5 second delay between requests (randomized)
Proxy:    None needed
Retry:    Exponential backoff on 429, max 3 retries
```

Level 2 — JS-rendered (Homebook, Oferteo):

```
Tool:     Playwright (headless Chrome)
Plugin:   playwright-extra + stealth
Headers:  Real browser fingerprint (automatic)
Rate:     5-10 second delay, randomized (Gaussian)
Proxy:    Datacenter proxy if blocked (cheap, ~$1/GB)
Session:  Navigate homepage → search → paginate (human-like flow)
```

Level 3 — Heavy protection (Houzz):

```
Tool:     Playwright + stealth + ghost-cursor
Proxy:    Residential (Bright Data / SmartProxy, ~$15/GB)
Rate:     8-20 second delay, business hours only
Fingerprint: Randomized viewport, timezone, WebGL, canvas
Session:  Full browsing simulation with scroll + mouse movement
Fallback: If consistently blocked → skip, use other sources
```

5.3 Rate Limiting & Politeness

```
// Gaussian delay — more human-like than uniform random
function gaussianDelay(meanMs, stdDevMs) {
  const u1 = Math.random();
  const u2 = Math.random();
  const z = Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
  return Math.max(1000, Math.round(meanMs + z * stdDevMs));
}

// Per-domain rate tracker
// If a domain returns 429 or 403:
//   1. Stop immediately
//   2. Wait: 60s × 2^(attempt-1) + random jitter
```

```
//   3. After 3 failures → mark domain as "blocked", skip for 24h
//   4. After 3 blocked days → alert human, remove from rotation
```

---

6. Solving the HTML Context Clogging Problem

6.1 The Problem

When an LLM agent scrapes a webpage, the raw HTML floods its context window. A single page can be 200KB+ of HTML, scripts, ads, and navigation chrome. The useful data (a business name, phone number, address) might be 200 bytes buried inside.

If we feed HTML to the agent:

- Context fills up after 2-3 pages
- Agent starts hallucinating or losing earlier context
- Performance degrades, costs increase
- Prompt injection risk: malicious HTML comments or hidden text can manipulate the agent

6.2 The Solution: Agent Never Sees Raw HTML

The scraper is a **dedicated Node.js script**, not an LLM task. The agent orchestrates scraping (triggers jobs, reads results) but never processes HTML.

```
Architecture:

  Agent (Claude in container)
    │
    │  "Run scraper for Warszawa architects"
    │
    ▼
  scraper.js (Node.js script, no LLM)
    │
    ├── Fetch HTML (undici / Playwright)
    ├── Parse with Cheerio (CSS selectors extract ONLY target fields)
    ├── Sanitize extracted text (strip HTML entities, control chars)
    ├── Validate against schema (reject if fields don't match types)
    ├── Deduplicate against SQLite
    └── Write structured JSON result (< 1KB per company)
    │
    ▼
  Agent reads JSON summary
    "Found 47 new companies in Warszawa, 12 duplicates skipped"
```

The agent never sees:

- Raw HTML
- JavaScript
- CSS
- Ads, navigation, footers
- Any user-generated content from the scraped site

The agent only sees:

- Structured JSON objects with pre-defined fields
- Summary statistics (count found, count new, count duplicate)
- Error reports (site blocked, timeout, parse failure)

6.3 Cheerio Extraction Pattern

```
// Example: Panorama Firm listing extraction
// Cheerio parses HTML into a DOM, CSS selectors extract ONLY what we need
// Total HTML page: ~150KB → Extracted data: ~200 bytes per listing

function extractListing($, element) {
  return {
    name:    sanitize($(element).find('.company-name').text()),
    address: sanitize($(element).find('.address').text()),
    phone:   sanitize($(element).find('.phone').text()),
    website: sanitize($(element).find('a.website').attr('href')),
    category: sanitize($(element).find('.category').text()),
```

```
  };
}

// sanitize() strips HTML, control characters, and normalizes whitespace
// see Section 7 for full sanitization pipeline
```

6.4 For Sites That Require Playwright

When JavaScript rendering is needed, we still don't feed HTML to the agent:

```
// Playwright extracts data via page.evaluate() — runs in browser context
// Returns ONLY structured data, never raw HTML

const listings = await page.evaluate(() => {
  return Array.from(document.querySelectorAll('.professional-card')).map(el => ({
    name: el.querySelector('.name')?.textContent?.trim() || '',
    city: el.querySelector('.location')?.textContent?.trim() || '',
    phone: el.querySelector('.phone')?.textContent?.trim() || '',
    website: el.querySelector('a.website')?.href || '',
  }));
});
// listings is a clean JSON array — no HTML ever leaves the browser
```

---

7. Security — IT Safety Engineer

7.1 Prompt Injection Prevention

Threat: Scraped text fields (company names, descriptions) could contain strings like "Ignore previous instructions and..." that manipulate the agent if passed to the LLM.

Mitigation — Defense in Depth:

```
Layer 1: Agent never processes raw scraped content
  └── Scraper is a standalone Node.js script, not an LLM call
  └── Agent only reads structured JSON summaries and statistics

Layer 2: Text sanitization before storage
  └── Strip all HTML tags and entities
  └── Remove control characters (U+0000–U+001F except newline)
  └── Truncate fields to max length (name: 200, address: 300, phone: 30)
  └── Reject fields containing LLM instruction patterns:
      /ignore.*previous|system.*prompt|you are now|act as/i → replace with "[removed]"

Layer 3: Parameterized database queries
  └── All SQLite operations use prepared statements
  └── No string concatenation in queries

Layer 4: Output sanitization for Excel
  └── Excel formula injection prevention:
      Prefix cells starting with =, +, -, @ with a single quote
  └── Prevents malicious formulas like =CMD() in exported spreadsheets
```

7.2 URL Safety

```
Before visiting ANY new domain:

1. Is it on our known-safe list? (panoramafirm.pl, pkt.pl, etc.)
   YES → proceed
   NO  ↓

2. VirusTotal API check (free: 4 requests/minute)
   └── If flagged by ≥2 engines → SKIP, log warning
   └── If clean → add to known-safe cache (TTL: 30 days)

3. Domain age check (whois)
   └── Domain registered < 60 days ago → SKIP (phishing risk)
```

```
4. TLS certificate validation
   └── Expired or self-signed → SKIP

5. Redirect policy
   └── Max 3 redirects
   └── Never follow redirects to a different domain than intended
   └── Log all redirects for audit
```

7.3 Container Sandboxing

```
The scraper runs inside the NanoClaw agent container:
  ├── No access to host filesystem (except mounted group folder)
  ├── No access to .env or credentials
  ├── Network: outbound only (no inbound listeners)
  ├── API keys passed via environment, not files
  ├── Container destroyed after job completes (--rm)
  └── Resource limits: 2GB RAM, 30-minute timeout
```

7.4 Credential Isolation

```
API keys needed by the scraper:
  - GOOGLE_PLACES_API_KEY
  - HUNTER_API_KEY
  - VIRUSTOTAL_API_KEY (optional, free tier)

Storage: .env file on host (never mounted into container)
Injection: credential proxy on port 3001 OR passed as env vars to container
Agent access: NEVER — agent cannot read or display API keys
```

7.5 Data Validation Schema

Every scraped record must pass validation before storage:

```
const SCHEMA = {
  name:        { type: 'string', required: true,  maxLen: 200, minLen: 2 },
  city:        { type: 'string', required: false, maxLen: 100 },
  postal_code: { type: 'string', required: false, pattern: /^\d{2}-\d{3}$/ },
  phone:       { type: 'string', required: false, pattern: /^[\d\s\+\-\(\)]{7,20}$/ },
  email:       { type: 'string', required: false, pattern: /^[^\s@]+@[^\s@]+\.[^\s@]+$/ },
  website:     { type: 'string', required: false, pattern: /^https?:\/\/.+/ },
  nip:         { type: 'string', required: false, pattern: /^\d{10}$/ },
};

// Records failing validation are logged but not stored
// This prevents garbage data and injection payloads from entering the DB
```

---

8. Data Schema (SQLite)

```
CREATE TABLE companies (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  name            TEXT NOT NULL,
  slug            TEXT UNIQUE,
  category        TEXT,       -- 'architekt' | 'projektant_wnetrz' | 'dekorator'
  nip             TEXT,       -- Polish tax ID
  regon           TEXT,       -- Statistical number
  pkd_code        TEXT,       -- '71.11.Z' or '74.10.Z'
  voivodeship     TEXT,       -- województwo
  city            TEXT,
  postal_code     TEXT,
  address         TEXT,
  phone           TEXT,
  website         TEXT,
  email           TEXT,       -- company email (biuro@, kontakt@, info@)
  google_rating   REAL,
```

```sql
  review_count    INTEGER,
  instagram       TEXT,
  facebook        TEXT,
  source          TEXT,       -- comma-separated: 'google,panorama,izba'
  quality_score   INTEGER DEFAULT 0,
  first_seen      TEXT,
  last_updated    TEXT
);

CREATE TABLE contacts (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  company_id      INTEGER REFERENCES companies(id),
  first_name      TEXT,
  last_name       TEXT,
  title           TEXT,       -- 'Właściciel', 'Główny Projektant'
  email           TEXT UNIQUE,
  email_verified  INTEGER DEFAULT 0,
  phone           TEXT,
  source          TEXT,
  first_seen      TEXT,
  last_verified   TEXT
);

CREATE TABLE scrape_log (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  source          TEXT NOT NULL,
  query           TEXT,
  city            TEXT,
  records_found   INTEGER DEFAULT 0,
  records_new     INTEGER DEFAULT 0,
  started_at      TEXT NOT NULL,
  completed_at    TEXT,
  status          TEXT DEFAULT 'running',
  error           TEXT
);

CREATE TABLE url_safety_cache (
  domain          TEXT PRIMARY KEY,
  status          TEXT,       -- 'safe' | 'blocked' | 'unknown'
  checked_at      TEXT,
  vt_score        TEXT        -- VirusTotal detection ratio
);

CREATE UNIQUE INDEX idx_slug ON companies(slug);
CREATE INDEX idx_city ON companies(city);
CREATE INDEX idx_category ON companies(category);
CREATE INDEX idx_voivodeship ON companies(voivodeship);
CREATE INDEX idx_quality ON companies(quality_score);
```

Deduplication

Slug = normalized name + postal code:

```
 "Studio Projektowe Wnętrza Sp. z o.o."  +  "01-234"
→ "studio-projektowe-wnetrza-01234"

Normalization:
1. Lowercase
2. Strip: sp. z o.o., s.a., s.c., sp.j., s.k., sp.k.
3. Transliterate: ą→a  ć→c  ę→e  ł→l  ń→n  ó→o  ś→s  ź→z  ż→z
4. Remove punctuation
5. Collapse whitespace → hyphens
6. Append postal code (digits only)
```

When duplicate found: merge sources, keep richest data (most fields filled).

Quality Score (0-100)

Factor	Points
Has verified email	+25

Has unverified email	+10
Has phone	+15
Has website (responds 200)	+15
Google rating ≥ 4.0	+10
Has ≥ 10 reviews	+5
Has social profiles	+5 per platform (max 10)
Professional association member	+10
Multiple sources confirm data	+5 per extra source (max 10)

---

9. Excel Export Format

Filename convention

```
leads_PL_{category}_{region}_{date}.xlsx
leads_PL_architects_warszawa_2026-03-21.xlsx
leads_PL_all_nationwide_2026-03-21.xlsx
```

Sheet 1: Firmy (Companies)

Nazwa	Kategoria	Miasto	Województwo	Kod pocztowy	Adres	Telefon	Email	WWW	NIP	Google Ocena	Liczba opinii	Instagram	Jakość	Źródło

Sheet 2: Kontakty (Contacts)

Firma	Imię	Nazwisko	Stanowisko	Email	Telefon	Zweryfikowany

Sheet 3: Statystyki (Summary)

Metric	Value
Total companies	
With email	
With phone	
Average quality score	
Sources used	
Export date	
Filter applied	

Export filters (via Telegram command)

```
@TataNano export all                    — everything
@TataNano export architects warszawa    — by category + city
@TataNano export designers małopolskie  — by category + voivodeship
@TataNano export quality 60+            — minimum quality score
@TataNano export email-ready            — only verified email records
```

Excel safety

- Cells starting with `=`, `+`, `-`, `@` are prefixed with `'` to prevent formula injection
- All strings are explicitly typed as text, not auto-detected
- File generated with `exceljs` library (npm), not manual CSV

---

10. Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    Telegram Chat                         │
│   "@TataNano scrape warszawa architects"                 │
│   "@TataNano export designers kraków"                    │
│   "@TataNano scraper status"                             │
└──────────────────────┬───────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│              NanoClaw Agent Container                    │
│                                                          │
│   Agent (Claude) receives command                        │
│     │                                                    │
│     ├── Understands intent                               │
│     ├── Calls scraper script (bash tool)                 │
│     ├── Reads JSON result (never HTML)                   │
│     └── Reports summary to user                          │
│                                                          │
│   ┌──────────────────────────────────────────────┐       │
│   │  Scraper Scripts (Node.js, no LLM)           │       │
│   │                                              │       │
│   │  scraper/                                    │       │
│   │  ├── index.ts          — CLI entry point     │       │
│   │  ├── google-places.ts  — Google Places API   │       │
│   │  ├── panorama-firm.ts  — panoramafirm.pl     │       │
│   │  ├── izba-arch.ts      — izbaarchitektow.pl  │       │
│   │  ├── pkt.ts            — pkt.pl yellow pages │       │
│   │  ├── regon.ts          — GUS/Regon API       │       │
│   │  ├── website-email.ts  — extract email from  │       │
│   │  │                       company websites    │       │
│   │  ├── hunter.ts         — Hunter.io API       │       │
│   │  ├── url-safety.ts     — VirusTotal + whois  │       │
│   │  ├── sanitizer.ts      — text cleaning       │       │
│   │  ├── dedup.ts          — slug generation     │       │
│   │  ├── db.ts             — SQLite operations   │       │
│   │  ├── validator.ts      — schema validation   │       │
│   │  └── excel-export.ts   — XLSX generation     │       │
│   └──────────────────────────────────────────────┘       │
│                                                          │
│   SQLite: /workspace/group/leads.db                      │
│   Exports: /workspace/group/exports/*.xlsx               │
└──────────────────────────────────────────────────────────┘
```

Data Flow

```
Source (API/Website)
  │
  ▼
Fetch (undici / Playwright)
  │
  ▼
Parse (Cheerio CSS selectors / page.evaluate)
  │    ↳ Only target fields extracted — no raw HTML retained
  ▼
Sanitize (strip HTML, control chars, truncate, injection filter)
  │
  ▼
Validate (schema check — reject malformed records)
  │
  ▼
Deduplicate (slug match against SQLite)
  │
  ├── New → INSERT
  └── Existing → MERGE (enrich with new source data)
  │
  ▼
SQLite (leads.db)
  │
  ▼ (on export command)
Excel (exceljs → .xlsx with formula injection protection)
```

```
  │
  ▼
Telegram (send file to user)
```

---

11. Build Phases

Phase 1: Foundation + Google Places (Week 1-2)

- SQLite schema + dedup logic + slug generator
- Sanitizer + validator modules
- Google Places API crawler (all 40 cities × 7 queries)
- Excel exporter with formula injection protection
- Telegram command integration (scrape, export, status)
- **Expected yield: 3,000-5,000 companies**
- **Cost: ~$140 (Google API)**

Phase 2: Polish Directories (Week 3-4)

- Panorama Firm scraper (Cheerio, simple HTML)
- Pkt.pl scraper
- Izba Architektów member list scraper
- SARP + SAW member directories
- Regon/GUS API integration (enrich with NIP, PKD, legal form)
- URL safety module (VirusTotal cache)
- Cross-reference and merge with Phase 1 data
- **Expected yield: +2,000-4,000 companies**
- **Cost: ~$0 (all free sources)**

Phase 3: Email Discovery (Week 5-6)

- Company website contact page scraper (targeted, Cheerio)
- Hunter.io domain search for remaining
- Apollo.io enrichment for decision-maker contacts
- Email verification pass
- Quality score calculation
- **Expected yield: emails for 60-70% of companies**
- **Cost: ~$49 (Hunter) + $0 (Apollo free tier)**

Phase 4: JS-Heavy Sites + Maintenance (Week 7-8)

- Playwright scrapers for Homebook.pl, Oferteo.pl
- Stealth + proxy setup for Houzz.pl (if worthwhile)
- Instagram business profile enrichment
- Scheduled monthly re-scrape (NanoClaw task scheduler)
- Quarterly email re-verification
- **Ongoing cost: ~$95/month**

---

12. Telegram Commands

```
Scraping:
  @TataNano scrape {city} {category}
  @TataNano scrape all cities
  @TataNano scrape panorama-firm warszawa
  @TataNano scraper status

Export:
  @TataNano export all
  @TataNano export architects warszawa
  @TataNano export email-ready
  @TataNano export quality 70+

Info:
  @TataNano leads count
  @TataNano leads stats
  @TataNano leads search "studio xyz"
```

13. Cost Summary

Phase 1 (one-time)

Item	Cost
Google Places API (~8,000 requests)	~$140
Hunter.io Starter (Phase 3)	$49
exceljs (npm)	Free
Total MVP	**~$190**

Monthly (at scale, after all phases)

Item	Monthly
Google Places (new listings check)	~$30
Hunter.io (ongoing lookups)	$49
Apollo.io (free tier)	$0
Datacenter proxy (if needed)	~$10
Total ongoing	**~$89/month**

No residential proxies needed for Polish sources = major cost saving vs DACH.

---

14. Expected Results

Phase	Companies	With Email	With Phone	Unique to Phase
Phase 1	3,000-5,000	~30%	~70%	3,000-5,000
Phase 2	6,000-8,000	~40%	~80%	+2,000-3,000
Phase 3	6,000-8,000	~65%	~80%	(enrichment)
Phase 4	7,000-9,000	~70%	~85%	+1,000-2,000

Polish market total: ~15,000-20,000 active interior design / architecture firms. This system should capture 40-50% of the addressable market.

---

15. Risk Register

Risk	Likelihood	Impact	Mitigation
Site changes HTML structure	High	Medium	CSS selectors are fragile. Each scraper has a health check — if extraction returns 0 results, alert human. Selectors stored in config, not hardcoded
IP blocked by directory	Medium	Low	Polish sites rarely block. If they do: add cheap datacenter proxy ($10/mo). Multiple sources mean losing one isn't fatal
Google Places API cost spike	Low	Medium	Budget cap per day. Monitor in scrape_log. Alert if >$10/day
Scraped data contains prompt injection	Low	High	Agent never sees raw scraped content. Sanitizer strips LLM instruction patterns. Validated against strict schema
Excel file used for phishing	Low	High	Formula injection prevention (prefix =,+,-,@ with quote). Files only sent to registered Telegram chat
Stale data in database	Medium	Medium	Monthly re-scrape. Quality score penalizes old data (-2 points/month). Contacts without engagement after 6 months flagged for review